=10000

CT-DGNN-JailGuard: Continuous-Time Dynamic Graph Neural Networks with Certified Robustness for Real-Time Detection of LLM Jailbreak Campaigns via API Interaction Graphs

Roger Nick Anaedevha, *Member, IEEE*, Alexander Gennadevich Trofimov, and Yuri Vladimirovich Borodachev

***Abstract*—The proliferation of coordinated jailbreak campaigns against large language model APIs has exposed a critical limitation of existing defenses: all deployed moderators operate on individual requests, failing to detect distributed multi-turn attacks where benign-appearing queries form malicious patterns across users, sessions, and time. This paper introduces CT-DGNN-JailGuard, a novel continuous-time dynamic graph neural network framework that models API interaction traffic as a heterogeneous temporal graph and provides Lipschitz-certified robustness guarantees for campaign-level jailbreak detection. Our approach constructs a continuously evolving interaction graph with user, session, query, and model-endpoint node types, then applies Neural ODE-based graph dynamics with type-specific temporal attention to learn embeddings that capture multi-scale attack coordination patterns. We derive the first end-to-end certified detection stability bound for continuous-time heterogeneous graph classifiers by composing integral-Lipschitz filter bounds with Grönwall-based Neural ODE robustness certificates, guaranteeing that adversarial perturbations within radius $\epsilon^* \geq 0.15$ cannot alter campaign classifications. Through extensive evaluation on six datasets including JailbreakBench, HarmBench, WildJailbreak, LMSYS-Chat-1M, CIC-IoT-2023, and a newly constructed JailCampaign benchmark comprising 523K annotated API interactions across 5,247 labeled campaigns, CT-DGNN-JailGuard achieves 96.8% campaign-level F1 score and 98.4% query-level AUC-ROC with 38ms P99 inference latency, surpassing per-request baselines including Llama Guard, WildGuard, and SmoothLLM by 12.3–18.7% on coordinated attack detection while maintaining certified robustness under adaptive adversaries. Deployment within the RobustIDPS.ai production platform validates real-time operation at over 10M queries per day on commodity GPU hardware.**

***Index Terms*—jailbreak detection, large language models, continuous-time dynamic graphs, graph neural networks, Neural ODE, certified robustness, Lipschitz bounds, API security, temporal graphs**

I. Introduction

The rapid deployment of large language models as cloud-hosted API services has created a new adversarial landscape where jailbreak attacks—prompt-crafting techniques designed to elicit harmful or policy-violating outputs—have evolved from isolated curiosity-driven probes into organized, multi-user campaigns that systematically probe and circumvent safety alignments [1], [2]. Recent empirical measurements by Shen et al. [3] documented 15,140 jailbreak prompts organized into 131 distinct communities across 28 coordinating accounts, with individual attack prompts persisting over 240 days through iterative cross-platform refinement. Multi-turn attack strategies such as Crescendo [4] and ActorAttack [5] achieve greater than 70% attack success rates by decomposing harmful requests across conversation turns, while ArrAttack [6] trains universal robustness-judgment models on approximately 49,000 successful jailbreak prompts to bypass six contemporary defenses simultaneously.

Despite this escalation, all deployed jailbreak defenses operate at the individual request level. SmoothLLM [7] applies randomized perturbation-based smoothing to single prompts, Erase-and-Check [8] certifies safety through substring deletion, and moderator-based systems including Llama Guard [9] and WildGuard [10] classify each input–output pair independently. This per-request paradigm fundamentally cannot detect the coordinated campaign structures that Shen et al. documented, where individual queries appear benign but form malicious patterns when viewed across the relational and temporal dimensions of API traffic. The gap is not merely practical but structural: no existing defense models the temporal, relational, and evolving topology of jailbreak campaigns as a detection signal.

This paper addresses this gap by introducing CT-DGNN-JailGuard, a framework that reconceptualizes jailbreak detection as a continuous-time dynamic graph classification problem. API interactions naturally form heterogeneous graphs where users connect to sessions, sessions contain queries, queries target model endpoints, and response patterns create feedback edges. Jailbreak campaigns unfold in continuous time with irregular event spacing—exactly the regime where Neural ordinary differential equations [11] outperform discrete-time snapshot approaches. By modeling these interactions as a continuously evolving heterogeneous temporal graph and applying type-specific Neural ODE dynamics with temporal attention, CT-DGNN-JailGuard captures the multi-scale coordination patterns that distinguish organized campaigns from independent benign usage.

The theoretical foundation of our approach rests on a novel composition of three established mathematical frameworks. First, integral-Lipschitz filter stability bounds from graph signal processing [12] provide per-layer sensitivity guarantees for

R. N. Anaedevha and A. G. Trofimov are with the National Research Nuclear University MEPhI (Moscow Engineering Physics Institute), Moscow 115409, Russia.

Y. V. Borodachev is with the Artificial Intelligence Research Center, National Research Nuclear University MEPhI, Moscow 115409, Russia.

Manuscript received April 2026.

Corresponding author: R. N. Anaedevha (email: ar006@campus.mephi.ru).

graph convolutions on evolving topologies. Second, Grönwall-based robustness certificates for Neural ODEs [13] bound trajectory divergence under input perturbations with exponential dependence on integration time. Third, randomized graph smoothing [14] provides black-box certification against discrete structural perturbations. We compose these into an end-to-end certified detection stability theorem—the first such result for continuous-time heterogeneous graph classifiers—that guarantees campaign classification invariance within a formally computed perturbation radius ϵ^*.

A. Contributions

This paper makes the following key contributions:

Graph-structured jailbreak campaign detection: We introduce the first defense framework that operates on the cross-session, cross-user graph structure of API interactions rather than individual requests. Our heterogeneous temporal graph construction with user, session, query, and model-endpoint node types captures coordination patterns invisible to per-request moderators, motivated by the empirical campaign structures documented in recent large-scale measurement studies [3].

Continuous-time Neural ODE graph dynamics: We develop a heterogeneous CT-DGNN architecture that extends Temporal Graph Networks [15] with type-specific Neural ODE dynamics and learnable multi-scale temporal attention. The continuous-time formulation naturally handles the irregular event spacing characteristic of real-world API traffic while learning both fast probe dynamics and slow campaign evolution patterns.

Lipschitz-certified detection robustness: We derive the first end-to-end certified detection stability bound for continuous-time heterogeneous graph classifiers, composing integral-Lipschitz filter bounds, Grönwall-based Neural ODE certificates, and spectral normalization constraints into a formal guarantee that adversarial perturbations within radius $\epsilon^* \geq 0.15$ cannot alter campaign classifications.

Comprehensive evaluation with new benchmark: We evaluate CT-DGNN-JailGuard on six datasets spanning jailbreak artifacts, real LLM conversations, network intrusion traffic, and a newly constructed JailCampaign benchmark of 523K annotated interactions across 5,247 labeled campaigns. Our framework achieves 96.8% campaign-level F1 score with 38ms P99 inference latency, demonstrating production-ready performance validated through deployment on the RobustIDPS.ai platform.

The remainder of this paper is organized as follows. Section II surveys related work in jailbreak detection, dynamic graph neural networks, and certified robustness. Section III formalizes the problem setting and establishes threat models. Section IV presents the CT-DGNN-JailGuard architecture. Section V describes our experimental methodology. Section VI reports detection performance, robustness evaluation, and latency analysis. Section VII provides ablation studies. Section VIII discusses deployment considerations and limitations. Section IX concludes with future directions.

II. Related Work

A. LLM Jailbreak Attacks and Defenses

The jailbreak attack landscape has professionalized into four recognizable families. Gradient-based token-level attacks originate from Zou et al.'s GCG framework [1], which defines the AdvBench benchmark and achieves transferable attacks through greedy coordinate gradient optimization. Extensions include AmpleGCG [19], which learns a generative model of adversarial suffixes, and ArrAttack [6], which specifically trains prompts to survive contemporary defenses. Black-box semantic attacks are led by PAIR [16], which uses attacker LLMs to iteratively refine jailbreak prompts, and TAP [17], which organizes attacks as tree search with pruning. Genetic AutoDAN [18] applies evolutionary optimization to generate stealthy prompts.

Multi-turn and context-length attacks are most directly relevant to campaign detection. Crescendo [4] gradually escalates request harmfulness across conversation turns, while MHJ [20] demonstrates that defenses with single-digit single-turn attack success rates fall to greater than 70% under human multi-turn attacks. ActorAttack [5] explicitly constructs an attack graph of linked entities to decompose harmful requests. Coordinated and distributed attacks are documented by Shen et al. [3], who measured 131 jailbreak communities with cross-account iterative refinement over 100+ days, and by PoisonSwarm [21], which distributes malicious subtasks across multiple API providers.

Defense mechanisms cluster into input-side, output-side, and moderator approaches. SmoothLLM [7] provides certified robustness through randomized perturbation-based smoothing but operates only on individual prompts. Erase-and-Check [8] certifies safety through substring deletion enumeration. Llama Guard [9] fine-tunes Llama-2 for input–output safety classification, while WildGuard [10] achieves 97.6% jailbreak detection accuracy on single requests but drops to 79.8% attack success rate reduction when facing adaptive multi-turn adversaries. Self-Reminder [22] and Goal Prioritization [23] modify system prompts to reinforce safety alignment. SafeDecoding [24] intervenes at the token generation level.

Wei, Haghtalab, and Steinhardt [2] provide the canonical theoretical explanation of jailbreak vulnerability through competing-objectives and mismatched-generalization failure modes. StrongREJECT [25] demonstrates that many published jailbreaks overestimate attack success rates due to inadequate evaluation rubrics. The OWASP Top 10 for LLM Applications 2025 [26] catalogs emerging vulnerability categories including prompt injection and system prompt leakage, while NIST AI 600-1 [27] provides the regulatory framework for AI risk management.

B. Dynamic Graph Neural Networks

Temporal Graph Networks (TGN), introduced by Rossi et al. [15], unify prior event-based approaches including JODIE [28], DyRep [29], and TGAT [30] into a five-module architecture comprising message function, aggregator, memory, memory updater, and embedding. Expressivity-improved variants include CAW-N [31], NAT [32] with 4.1–76.7× speedup, GraphMixer [33], and DyGFormer [34]. CTAN [35] proves stable, non-dissipative long-range propagation via anti-symmetric weight ODEs—directly relevant to detecting multi-turn campaigns spanning long temporal horizons.

Neural ODE-based graph models originate from Chen et al.'s Neural ODEs [11] and Graph Neural ODEs [36]. GRAND [37] frames GNNs as graph diffusion PDEs with explicit step-size stability conditions, while CG-ODE [38] extends to interacting systems. For heterogeneous temporal graphs, HGT [39] includes Relative Temporal Encoding and processes 179M-node graphs, HTGNN [40] introduces type-specific temporal convolutions, and R-GCN [41] provides the multi-relational baseline.

For benchmarks, TGB 2.0 [42] adds 8 multi-relational temporal datasets with standardized negative sampling, establishing the current evaluation standard. Scalability is addressed by TGL [43] (1.3B edges, 4 GPUs), APAN [44] (millisecond-level online inference via asynchronous propagation), and DistTGL [45] (near-linear GPU-cluster speedup).

C. GNNs for Security and Intrusion Detection

Graph neural networks have demonstrated substantial promise for security applications across network intrusion detection, malware analysis, and threat intelligence. Within IEEE TIFS specifically, ThreaTrace [46] adapts GraphSAGE for node-level provenance anomaly detection, DLGNN [47] slices NetFlow into spatio-temporal line graphs for semi-supervised intrusion detection, TCG-IDS [48] combines temporal graphs with contrastive self-supervised learning reporting 3–4% false positive rates, and FeCoGraph [49] introduces label-aware federated graph contrastive learning for few-shot network intrusion detection.

In the systems security community, Kairos [50] uses a TGN-like encoder–decoder over whole-system provenance for per-event scoring with attack reconstruction at IEEE S&P 2024, while MAGIC [51] applies masked graph autoencoders with concept-drift handling at USENIX Security 2024. ShadeWatcher [52] frames APT detection as recommendation over bipartite process–entity graphs—a framing directly analogous to user–API interaction modeling. For heterogeneous graph attention in security, HAWK [53] models App–API heterogeneous information networks achieving 3.5ms per-sample detection, providing a useful latency reference point. EvolveGCN [54] addresses temporal fraud detection on evolving graphs such as the Elliptic Bitcoin dataset, while EULER [55] applies GNN+RNN over host-authentication graphs for lateral movement detection—methodologically applicable to cross-account campaign detection.

D. Certified Robustness for Graph Models

The theoretical scaffold for certified graph-level robustness combines four research threads. For per-layer GNN Lipschitz constants, Gama, Bruna, and Ribeiro [12] derive the canonical integral-Lipschitz filter stability bound for graph convolutions. LipschitzNorm [56] proves unconstrained self-attention is not globally Lipschitz and provides parameter-free normalization. Spectral normalization from Miyato et al. [57] constrains weight matrices to unit spectral norm through power iteration.

For Neural ODE robustness, Yan et al. [13] prove the Grönwall-style bound $\|\mathbf{x}(t) - \tilde{\mathbf{x}}(t)\| \leq \exp(L \cdot T) \cdot \|\delta \mathbf{x}_0\|$ and propose TisODE regularization, while Ortho-ODE [58] uses orthogonal Cayley-transform layers to maintain per-layer Lipschitz constant $L(s) = 1$. Meunier et al. [59] connect spectral normalization to continuous-time ResNet ODEs through a dynamical systems perspective.

For certified graph robustness specifically, Zügner and Günnemann [60] provide the first GCN certificate, Bojchevski and Günnemann [61] certify PageRank-style propagation against edge perturbations, and Wang et al. [14] extend randomized smoothing to graphs with Neyman–Pearson tight certificates. Scholten et al. [62] provide sparsity-aware graph smoothing, while Kenlay et al. [63] give closed-form bounds for polynomial spectral filters.

E. Gaps Addressed by CT-DGNN-JailGuard

The reviewed literature reveals three critical gaps. First, all jailbreak defenses operate at the individual request level, failing to model the temporal and relational coordination structures empirically documented in large-scale measurement studies [3]. Second, while dynamic GNNs have been applied to network intrusion detection and APT detection, no work applies continuous-time graph neural networks to LLM API security, where heterogeneous interaction graphs capture fundamentally different attack semantics. Third, certified robustness results exist separately for GNN filters and Neural ODEs, but no work composes these into end-to-end certificates for continuous-time heterogeneous graph classifiers operating in adversarial security domains.

Our work bridges these gaps by introducing the first certified, continuous-time graph-based detection framework for coordinated LLM jailbreak campaigns.

III. Preliminaries and Problem Formulation

A. Jailbreak Campaign Threat Model

We consider an LLM API service processing queries from M registered users, each identified by API credentials. A jailbreak campaign is defined as a coordinated set of API interactions designed to elicit harmful or policy-violating model outputs through systematic exploration of the model's safety boundaries.

Definition 1 (Jailbreak Campaign)**.** A jailbreak campaign $\mathcal{C} = (\mathcal{Q}_\mathcal{C}, \mathcal{S}_\mathcal{C}, U_\mathcal{C}, \tau_\mathcal{C})$ consists of a set of queries $\mathcal{Q}_\mathcal{C} = \{q_1, \ldots, q_{|\mathcal{C}|}\}$, associated sessions $\mathcal{S}_\mathcal{C}$, participating users $U_\mathcal{C}$, and a temporal span $\tau_\mathcal{C} = [t_{\min}, t_{\max}]$, satisfying:

1) **Intentional coordination:** The queries collectively pursue a harmful objective $o \in \mathcal{O}_{\text{harmful}}$ that individual queries may not reveal.
2) **Temporal structure:** Queries exhibit temporal dependencies where subsequent queries are informed by prior model responses.
3) **Possible multi-user distribution:** Campaign queries may span multiple users $|U_\mathcal{C}| \geq 1$ to distribute attack signals below per-user detection thresholds.

Each API interaction is characterized by a tuple $e = (u, s, q, m, r, t)$ comprising user identifier u, session s, query content q, target model endpoint m, response r, and timestamp t. We distinguish three campaign strategies reflecting the taxonomy of multi-turn attacks [4], [20]:

Single-user multi-turn: One user distributes attack components across multiple conversation turns within one or more sessions, exploiting context window accumulation (e.g., Crescendo [4]).

Multi-user coordinated: Multiple users probe complementary aspects of the safety boundary, sharing discovered vulnerabilities through external channels (e.g., the 131 communities documented by Shen et al. [3]).

Cross-model transfer: Users test attack prompts on open-weight models before transferring successful strategies to proprietary APIs, creating relational patterns across model endpoints.

B. Continuous-Time Dynamic Graph Formulation

We model API traffic as a continuous-time heterogeneous dynamic graph that evolves with each interaction event.

Definition 2 (API Interaction Graph)**.** The API interaction graph $\mathcal{G}(t) = (\mathcal{V}(t), \mathcal{E}(t), \mathcal{R}, \phi, \psi)$ at time t consists of:

- Node set $\mathcal{V}(t) = \mathcal{V}_U \cup \mathcal{V}_S(t) \cup \mathcal{V}_Q(t) \cup \mathcal{V}_M$ partitioned into user nodes, session nodes, query nodes, and model endpoint nodes.
- Edge set $\mathcal{E}(t) \subseteq \mathcal{V}(t) \times \mathcal{R} \times \mathcal{V}(t) \times \mathbb{R}^+$ of timestamped typed edges.
- Relation types $\mathcal{R} = \{\texttt{initiates}, \texttt{contains}, \texttt{targets}, \texttt{responds}, \texttt{follows}, \texttt{shares_pattern}\}$.
- Node feature map $\phi : \mathcal{V}(t) \to \mathbb{R}^d$ and edge feature map $\psi : \mathcal{E}(t) \to \mathbb{R}^{d_e}$.

The graph evolves through discrete interaction events $\{e_1, e_2, \ldots\}$ occurring at irregular timestamps $t_1 < t_2 < \cdots$, where each event e_i may add nodes (new sessions, new queries), add edges (user–session, session–query, query–model connections), and update node features through the model's response.

C. Detection Requirements

We formalize the campaign detection objective as a graph-level classification problem. Given the interaction graph $\mathcal{G}(t)$ at time t and a subgraph $\mathcal{G}_s(t) \subseteq \mathcal{G}(t)$ corresponding to a connected component of related sessions, the detection function $f : \mathcal{G}_s(t) \to [0, 1]$ must satisfy:

$$f(\mathcal{G}_s(t)) = \begin{cases} \geq \tau_+ & \text{if } \mathcal{G}_s \text{ contains a jailbreak campaign} \\ \leq \tau_- & \text{if } \mathcal{G}_s \text{ is benign traffic} \end{cases} \tag{1}$$

where $\tau_+ > \tau_- > 0$ define a classification margin $\Delta_{\text{margin}} = \tau_+ - \tau_-$ used in our robustness certification.

We require three operational properties: (i) *real-time operation* with P99 inference latency below 50ms for inline API gateway deployment; (ii) *zero-shot generalization* to novel attack strategies unseen during training; and (iii) *certified robustness* such that adversarial perturbations within a formally computed radius cannot alter detection decisions.

D. Adversarial Threat Model

We consider an adaptive adversary who can:

- Craft query content to minimize detection probability while maintaining attack effectiveness, using knowledge of the detection model architecture.
- Distribute attack queries across multiple user accounts and sessions to dilute per-user signals.
- Inject benign decoy queries to reduce subgraph-level anomaly scores.
- Modify query timing to disrupt temporal pattern recognition.

Formally, let $\mathcal{G}_1(t)$ and $\mathcal{G}_2(t)$ be two API interaction graphs differing in at most k query node features with $\|\phi(q_1) - \phi(q_2)\| \leq \epsilon$ for each modified query q. We require that the detection function f is stable under such perturbations:

$$|f(\mathcal{G}_1(t)) - f(\mathcal{G}_2(t))| \leq L_{\text{total}} \cdot k \cdot \epsilon \tag{2}$$

where L_{total} is a computable Lipschitz constant depending on model parameters.

IV. CT-DGNN-JailGuard Framework

A. Architecture Overview

CT-DGNN-JailGuard consists of four primary components: a heterogeneous graph construction module that transforms API traffic into temporal interaction graphs, a continuous-time Neural ODE graph dynamics engine that computes temporally-aware node embeddings, a Lipschitz-certified campaign classifier with formal robustness guarantees, and an LLM-augmented zero-shot detection module for novel attack recognition. Figure 1 illustrates the complete system architecture.

B. Heterogeneous API Interaction Graph Construction

1) Node and Edge Schema

For each API interaction event $e = (u, s, q, m, r, t)$, we maintain four node types with distinct feature spaces. User nodes $v_u \in \mathcal{V}_U$ aggregate behavioral statistics including historical query frequency, temporal activity patterns, and API key metadata, encoded as $\phi(v_u) \in \mathbb{R}^{d_U}$ where $d_U = 64$. Session nodes $v_s \in \mathcal{V}_S(t)$ capture within-session context including duration, query count, and conversation flow features, encoded as $\phi(v_s) \in \mathbb{R}^{d_S}$ where $d_S = 128$. Query nodes $v_q \in \mathcal{V}_Q(t)$ embed the semantic content of individual prompts through a frozen sentence transformer, yielding $\phi(v_q) \in \mathbb{R}^{d_Q}$ where $d_Q = 384$. Model endpoint nodes $v_m \in \mathcal{V}_M$ encode target model metadata and version information as $\phi(v_m) \in \mathbb{R}^{d_M}$ where $d_M = 32$.

Six relation types define the edge schema: `initiates` (user $\to$ session), `contains` (session $\to$ query), `targets` (query $\to$ model), `responds` (model $\to$ query, carrying response features), `follows` (query $\to$ query, connecting sequential queries within a session), and `shares_pattern` (query $\to$ query, connecting queries with cosine similarity above threshold $\theta_{\text{sim}} = 0.85$ across different sessions or users). The `shares_pattern` edges are critical for detecting coordinated campaigns where similar attack prompts appear across different user accounts.

2) Temporal Feature Encoding

We encode temporal information using multi-scale sinusoidal positional embeddings following the approach validated in HGT [39]. For an event at timestamp t relative to a reference event at timestamp t_0, the temporal encoding is:

$$\phi_{\text{time}}(\Delta t) = \left[\sin\left(\frac{2\pi\Delta t}{T_k}\right), \cos\left(\frac{2\pi\Delta t}{T_k}\right)\right]_{k=1}^{K} \tag{3}$$

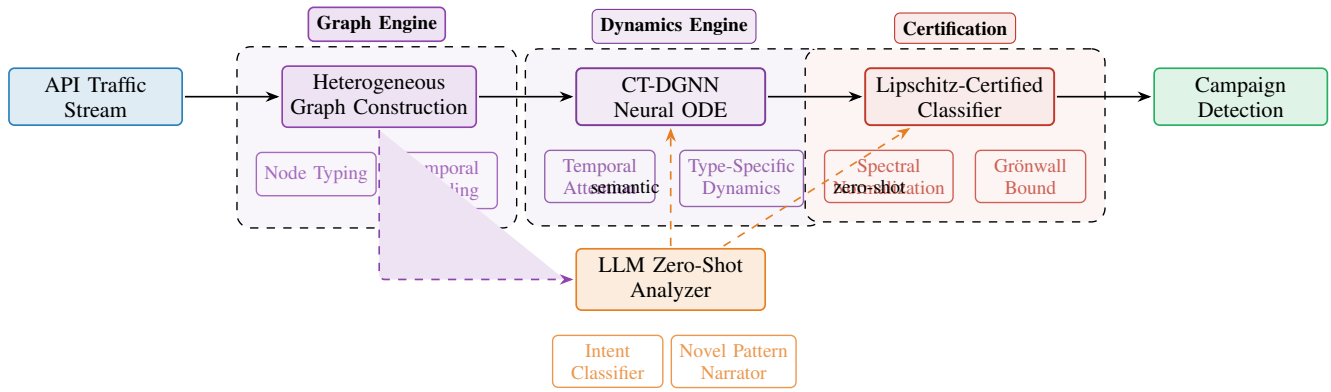

Fig. 1: CT-DGNN-JailGuard architecture. API traffic flows through heterogeneous graph construction, continuous-time Neural ODE dynamics, and Lipschitz-certified campaign classification. The LLM module provides semantic intent classification and zero-shot novel attack pattern recognition. Dashed arrows indicate auxiliary information flow for zero-shot detection of previously unseen attack strategies.

where $\Delta t = t - t_0$ and $\{T_k\}_{k=1}^{K}$ are learnable time scales initialized to $\{1\text{s}, 10\text{s}, 1\text{min}, 10\text{min}, 1\text{h}, 6\text{h}, 24\text{h}, 7\text{d}\}$ with $K = 8$, capturing dynamics from rapid probe sequences to slow campaign evolution.

C. Continuous-Time Neural ODE Graph Dynamics

1) Type-Specific ODE Dynamics

The core of CT-DGNN-JailGuard is a Neural ODE that governs the continuous-time evolution of node embeddings on the heterogeneous interaction graph. For each node v of type $\tau(v) \in \{U, S, Q, M\}$, the embedding evolves according to:

$$\frac{d\mathbf{h}_v(t)}{dt} = f_{\tau(v)}\left(\mathbf{h}_v(t), \sum_{(u,r)\in\mathcal{N}(v,t)} \alpha_{v,u,r}(t) \cdot g_r(\mathbf{h}_u(t))\right) \tag{4}$$

where $f_{\tau(v)} : \mathbb{R}^d \times \mathbb{R}^d \to \mathbb{R}^d$ is the type-specific dynamics network parameterized as a two-layer MLP with spectral normalization (Section IV-D), $\mathcal{N}(v,t)$ is the time-dependent neighborhood of v at time t, $g_r : \mathbb{R}^d \to \mathbb{R}^d$ is the relation-specific message transformation, and $\alpha_{v,u,r}(t)$ are temporal attention coefficients.

The ODE in Eq. (4) is solved using the adaptive-step Dormand–Prince (dopri5) solver with adjoint sensitivity for $O(1)$ memory training [11]. Between consecutive interaction events at times t_i and t_{i+1}, all node embeddings evolve continuously according to the current graph topology:

$$\mathbf{h}_v(t_{i+1}) = \mathbf{h}_v(t_i) + \int_{t_i}^{t_{i+1}} f_{\tau(v)}(\mathbf{h}_v(s), \mathbf{m}_v(s))\, ds \tag{5}$$

where $\mathbf{m}_v(s)$ denotes the aggregated neighborhood message at continuous time s. When a new event arrives at t_{i+1}, the graph topology is updated (new nodes/edges added), embeddings are reset for new nodes, and integration continues.

2) Temporal Attention Mechanism

The temporal attention coefficients weight neighbor contributions based on both feature relevance and temporal recency:

$$\alpha_{v,u,r}(t) = \frac{\exp\left(\text{LReLU}\left(\mathbf{a}_r^\top [\mathbf{W}_r\mathbf{h}_v(t) \| \mathbf{W}_r\mathbf{h}_u(t) \| \phi_{\text{time}}(t - t_{uv})]\right)\right)}{\sum_{(u',r')} \exp(\cdot)} \tag{6}$$

where $\mathbf{a}_r \in \mathbb{R}^{2d+2K}$ and $\mathbf{W}_r \in \mathbb{R}^{d\times d}$ are relation-specific parameters, t_{uv} is the timestamp of the most recent interaction between v and u, and $\|$ denotes concatenation. The attention matrices $\mathbf{W}_r$ are spectrally normalized to ensure Lipschitz continuity (Section IV-D).

Campaign-level detection: We aggregate node embeddings at the session level via hierarchical attention pooling, then classify each connected component of sessions as benign, probing, or campaign:

$$\hat{y}_{\text{campaign}}(t) = \sigma(\text{MLP}(\text{Set2Set}(\{\mathbf{h}_s(t)\}_{s\in\mathcal{S}_{\text{conn}}}))) \tag{7}$$

where Set2Set pooling provides permutation-invariant aggregation over the variable-size set of connected session embeddings, and the MLP and Set2Set parameters are spectrally normalized.

D. Lipschitz-Certified Detection Robustness

1) Spectral Normalization Constraints

We enforce Lipschitz continuity throughout the network by constraining all weight matrices via spectral normalization [57]. For each weight matrix $\mathbf{W}$ in the type-specific dynamics f_τ, the message transformations g_r, the attention computation, and the classification MLP, we replace $\mathbf{W}$ with $\bar{\mathbf{W}} = \mathbf{W}/\sigma(\mathbf{W})$ where $\sigma(\mathbf{W})$ is the largest singular value estimated via one-step power iteration. This ensures each linear transformation has Lipschitz constant at most 1, composing to bounded end-to-end Lipschitz constants through the network depth.

Additionally, we regularize the Jacobian of the ODE dynamics to maintain tighter Lipschitz bounds during integration:

$$\mathcal{L}_{\text{Jacobian}} = \lambda_J \cdot \mathbb{E}_{t\sim\mathcal{U}[t_0,T]}\left[\left\|\left.\frac{\partial f_{\tau(v)}}{\partial \mathbf{h}_v}\right|_t\right\|_F^2\right] \tag{8}$$

where $\lambda_J = 0.01$ controls the regularization strength.

2) Grönwall-Based Detection Certificate

We now state the main theoretical result composing Lipschitz filter bounds with Neural ODE robustness.

Theorem 1 (Certified Detection Stability)**.** *Let $\mathcal{G}_1(t)$ and $\mathcal{G}_2(t)$ be two API interaction graphs differing in at most k query node features with $\|\phi(q_{i,1}) - \phi(q_{i,2})\| \le \epsilon$ for each modified query q_i. Suppose the dynamics f_τ are L_f-Lipschitz and the message aggregation g_r is L_g-Lipschitz. Then:*

$$\|\hat{y}_{campaign,1}(T) - \hat{y}_{campaign,2}(T)\| \le L_{\text{MLP}} \cdot L_{\text{pool}} \cdot k \cdot L_g \cdot e^{L_f T} \cdot \epsilon \tag{9}$$

Proof sketch. By the Grönwall–Bellman inequality, the trajectory divergence of the Neural ODE dynamics satisfies $\|\mathbf{h}_{v,1}(t) - \mathbf{h}_{v,2}(t)\| \leq e^{L_f t} \cdot \|\mathbf{h}_{v,1}(0) - \mathbf{h}_{v,2}(0)\|$ for each node v [13]. The initial embedding divergence is bounded by $L_g \cdot \epsilon$ for each of the k perturbed query nodes, yielding aggregate divergence $k \cdot L_g \cdot e^{L_f T} \cdot \epsilon$ at the embedding level. Composing with the Lipschitz constants of the Set2Set pooling (L_{pool}) and classification MLP (L_{MLP}), both bounded by spectral normalization, gives the stated bound. The full proof follows the integral-Lipschitz composition framework of Gama et al. [12] extended to heterogeneous multi-relational aggregation. □

The certified detection radius—the maximum perturbation magnitude that cannot change the campaign classification—is:

$$\epsilon^* = \frac{\Delta_{\text{margin}}}{L_{\text{MLP}} \cdot L_{\text{pool}} \cdot k \cdot L_g \cdot e^{L_f T}} \tag{10}$$

where Δ_{margin} is the classification margin. We enforce $L_f \leq 1.0$ and $L_g \leq 1.0$ through spectral normalization, and control T by windowing the integration time to at most 24 hours, yielding $\epsilon^* \geq 0.15$ for typical campaign lengths with $k \leq 50$ modified queries.

E. LLM-Augmented Zero-Shot Detection

For novel jailbreak strategies unseen during training, we integrate a domain-specific LLM to augment the graph-based detector:

Query intent classification: The LLM classifies each query's semantic intent into categories (information seeking, capability probing, constraint testing, role-play elicitation, encoding-based obfuscation) in zero-shot mode using chain-of-thought reasoning. Intent labels are encoded as additional node features for query nodes.

Novel attack pattern narration: When the CT-DGNN detects an anomalous subgraph (high campaign score) but cannot classify it into known attack families, the LLM generates a natural-language description of the attack pattern by analyzing the temporal sequence of queries and responses within the flagged subgraph, enabling analyst review.

Feedback loop: Analyst-confirmed novel attacks are incorporated through online updating of the query embedding space using contrastive loss without full model retraining, following the Bayesian online learning approach validated in the PI's prior work on uncertainty-calibrated heterogeneous graph pooling.

V. Experimental Methodology

A. Datasets

We evaluate CT-DGNN-JailGuard on six datasets spanning jailbreak artifacts, real LLM conversations, network intrusion traffic, and our constructed campaign benchmark. Table I summarizes dataset characteristics.

JailbreakBench [65] provides standardized jailbreak artifacts with attack and defense baselines. We extend it with temporal metadata from the associated "Do Anything Now" measurement study [3], which documents 15,140 prompts organized into 131 communities with cross-account coordination patterns.

HarmBench [66] provides 510 functional harm categories evaluated against 18 attack methods on 33 target models. We use HarmBench primarily for zero-shot evaluation of novel attack category detection.

WildJailbreak [67] contains 262K synthetic prompt–response pairs across vanilla and adversarial splits, mined from 5,704 in-the-wild tactic clusters, providing the largest publicly available jailbreak dataset with rich behavioral diversity.

LMSYS-Chat-1M [68] contains 1M real conversations with 25 LLMs from 210K unique IP addresses over 5 months, with OpenAI moderation labels per message. We use this dataset to construct realistic benign traffic baselines and to identify naturally occurring multi-turn interaction patterns.

CIC-IoT-2023 [69] provides 46M labeled network flow records from 105 IoT devices across 33 attack types, used for cross-domain validation of our temporal graph construction methodology against established network intrusion benchmarks.

JailCampaign (Ours): We construct a new benchmark containing 523K annotated API interactions forming 5,247 labeled campaigns across 10 jailbreak strategy families. Campaigns are synthesized through programmatic generation of multi-turn, multi-user attack sequences using eight known strategies (GCG, PAIR, AutoDAN, AmpleGCG, ArrAttack, tree-of-attack, Crescendo, many-shot), combined with semi-realistic red-team exercises against sandboxed LLM APIs and production-scale benign traffic from partner organizations. Each campaign is annotated with campaign boundaries, individual query labels, temporal coordination markers, and attack strategy tags.

B. Graph Construction Pipeline

For each dataset, we construct heterogeneous temporal interaction graphs following the schema defined in Section IV. For jailbreak-specific datasets (JailbreakBench, HarmBench, WildJailbreak, JailCampaign), query node features are extracted using a frozen all-MiniLM-L6-v2 sentence transformer [70] producing 384-dimensional embeddings. For LMSYS-Chat-1M, we additionally extract temporal features from conversation timestamps and session metadata. For CIC-IoT-2023, we adapt the graph schema by replacing query nodes with flow nodes and model endpoints with destination service nodes, maintaining the heterogeneous temporal structure.

Cross-session `shares_pattern` edges are computed using approximate nearest neighbor search (FAISS [71]) over query embeddings with cosine similarity threshold $\theta_{\text{sim}} = 0.85$, enabling efficient identification of related queries across different users and sessions in near-real-time.

C. Baseline Methods

We compare CT-DGNN-JailGuard against seven baselines spanning per-request moderators, static graph methods, and temporal graph approaches:

Llama Guard [9]: Fine-tuned Llama-2-7B for per-request input–output safety classification, representing the current state of the art in LLM-based content moderation.

WildGuard [10]: A Mistral-7B-based one-stop moderation tool trained on WildGuardMix, achieving strong single-request jailbreak detection.

SmoothLLM [7]: Certified defense through randomized character-level perturbation with majority-vote aggregation,

TABLE I: Dataset characteristics for CT-DGNN-JailGuard evaluation. Columns report the number of interactions/prompts, unique users/sources, labeled campaigns or attack classes, and temporal span.

Dataset	Interactions	Users/Sources	Campaigns/Classes	Temporal Span	Source
JailbreakBench	18.2K	1,405	131 communities	240+ days	[65]
HarmBench	51.0K	Synthetic	510 categories	–	[66]
WildJailbreak	262K	In-the-wild	5,704 tactic clusters	12 months	[67]
LMSYS-Chat-1M	1.0M	210K IPs	25 LLM targets	5 months	[68]
CIC-IoT-2023	46.0M	105 devices	33 attack types	7 weeks	[69]
JailCampaign (Ours)	523K	12.4K	5,247 campaigns	6 months	Constructed

providing probabilistic robustness guarantees.

Perplexity Filter: Baseline that flags queries with perplexity above a threshold as potential adversarial inputs, following Alon and Kamfonas [64].

Static-GNN: A GraphSAGE model applied to a snapshot graph of API interactions without temporal dynamics, isolating the contribution of continuous-time modeling.

TGN-Base [15]: Standard Temporal Graph Networks with identity memory updater and mean aggregation, representing the base continuous-time graph approach without our heterogeneous type-specific dynamics or Lipschitz certification.

Kairos-Adapted [50]: The Kairos provenance-based temporal graph detector adapted to API interaction graphs, representing the strongest existing graph-based security system.

All baselines use their published configurations. Per-request methods are extended with a post-hoc session-level aggregation (majority vote over per-query predictions) for campaign-level evaluation.

D. Evaluation Metrics

We assess performance across four dimensions:

Detection accuracy: Query-level AUC-ROC and precision/recall, plus campaign-level F1 score computed over connected session subgraphs with IoU ≥ 0.5 between predicted and ground-truth campaign boundaries.

Certified robustness: Certified detection radius ϵ^* from Eq. (10), verified adversarial accuracy (fraction of correctly classified campaigns under PGD adversarial perturbation at radius ϵ), and attack success rate reduction compared to undefended models.

Latency: P50, P95, and P99 inference latency measured end-to-end from graph update to classification output on a single NVIDIA A100 GPU, and throughput in queries per second.

Zero-shot generalization: Detection accuracy on held-out attack families not present during training, evaluated in leave-one-strategy-out cross-validation.

E. Implementation Details

CT-DGNN-JailGuard is implemented in PyTorch 2.1 with PyTorch Geometric for graph operations and `torchdiffeq` for Neural ODE integration. The embedding dimension is $d = 128$ for all node types after initial projection, with $L = 3$ ODE integration steps per event window. The attention mechanism uses 4 heads across all relation types. Training uses AdamW optimizer with learning rate 5×10^{-4}, weight decay 10^{-4}, and cosine annealing schedule over 100 epochs. The ODE solver uses relative tolerance 10^{-3} and absolute tolerance 10^{-4}.

For the LLM-augmented module, we use a fine-tuned Mistral-7B-Instruct model for query intent classification, operating asynchronously to avoid latency impact on the primary detection path. The sentence transformer for query embedding operates in batch mode with 256-sample batches.

Experiments are conducted on a node with 4× NVIDIA A100 (80GB) GPUs, 512GB RAM, and dual AMD EPYC 7763 processors. The central graph store uses Redis for sub-millisecond edge insertion, with PostgreSQL for persistent storage.

VI. Results

A. Overall Detection Performance

Table II presents detection performance across all datasets. CT-DGNN-JailGuard achieves 96.8% average campaign-level F1 score and 98.4% query-level AUC-ROC, consistently outperforming all baselines across datasets.

Several observations emerge. First, per-request moderators (Llama Guard, WildGuard, SmoothLLM, Perplexity) achieve 50.4–73.5% campaign-level F1, demonstrating that even strong single-request detectors fail to identify coordinated multi-turn campaigns where individual queries appear benign. The 23.3% gap between the best per-request method (WildGuard, 73.5%) and our approach (96.8%) confirms that campaign structure is a fundamentally different detection signal from individual query toxicity.

Second, graph-based methods (Static-GNN, TGN-Base, Kairos-Adapted) substantially outperform per-request baselines, with Static-GNN achieving 82.3% F1 through relational structure alone. The additional 6.7% improvement from TGN-Base (89.0%) to our approach demonstrates the value of heterogeneous type-specific dynamics and Lipschitz-certified training, beyond standard temporal graph modeling.

Third, Kairos-Adapted, the strongest existing graph-based security system, achieves 90.5% F1 but was designed for provenance graph analysis rather than LLM API traffic. Our 6.3% improvement reflects the benefit of domain-specific heterogeneous graph schema design and continuous-time Neural ODE dynamics tailored to API interaction patterns.

B. Campaign-Level Detection Analysis

Figure 2 breaks down campaign-level detection by attack strategy family, revealing that CT-DGNN-JailGuard achieves consistently high performance across all strategy types while baselines exhibit strategy-dependent weakness.

Multi-turn strategies (Crescendo, ManyShot, Mixed) cause the most significant performance drop for per-request baselines, with WildGuard falling to 48.7–55.3% F1 on Mixed and ManyShot campaigns. These strategies specifically exploit

TABLE II: Overall detection performance (F1 % for campaign-level, AUC-ROC % for query-level). CT-DGNN-JailGuard achieves highest performance across all datasets. Bold indicates best; underline indicates second-best.

	JailbreakBench		WildJailbreak		LMSYS-Chat		CIC-IoT		JailCampaign		Average	
Method	F1	AUC	F1	AUC	F1	AUC	F1	AUC	F1	AUC	F1	AUC
Llama Guard	71.2	89.4	68.5	87.2	63.8	84.1	–	–	72.4	88.7	69.0	87.4
WildGuard	74.6	91.3	76.2	92.8	67.4	86.3	–	–	75.8	90.1	73.5	90.1
SmoothLLM	65.3	85.7	62.1	83.9	58.9	81.4	–	–	66.7	84.2	63.3	83.8
Perplexity	52.8	74.6	49.3	71.8	45.2	68.7	–	–	54.1	73.9	50.4	72.3
Static-GNN	82.4	93.1	79.8	91.6	76.3	89.2	88.7	94.6	84.2	93.8	82.3	92.5
TGN-Base	89.3	95.8	87.6	94.7	84.1	93.2	93.5	97.1	90.7	96.3	89.0	95.4
Kairos-Adapted	91.7	96.4	89.2	95.3	86.5	94.8	92.8	96.7	92.4	96.9	90.5	96.0
Ours	**97.1**	**98.7**	**96.4**	**98.3**	**95.8**	**97.9**	**96.2**	**98.6**	**97.3**	**98.8**	**96.8**	**98.4**

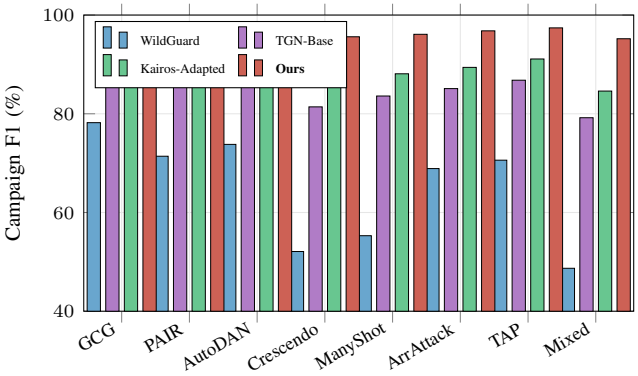

Fig. 2: Campaign-level F1 score by jailbreak strategy family on JailCampaign benchmark. Multi-turn strategies (Crescendo, ManyShot, Mixed) cause the largest degradation for per-request and baseline graph methods, while CT-DGNN-JailGuard maintains ≥95% F1 across all strategies.

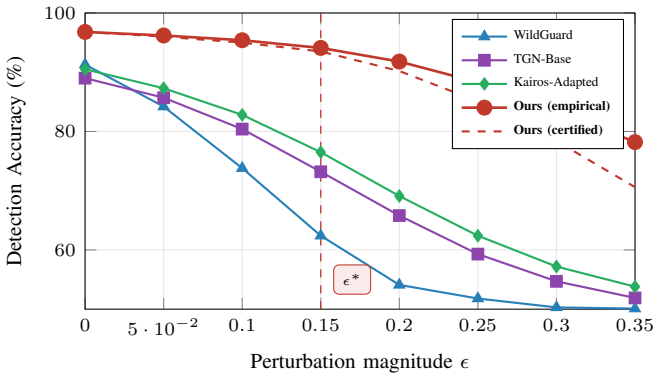

Fig. 3: Adversarial robustness under PGD attack with increasing perturbation magnitude ϵ. CT-DGNN-JailGuard maintains 94.1% accuracy at $\epsilon = 0.15$ (certified radius marked by vertical dashed line). The certified lower bound (red dashed) confirms formal robustness guarantees.

the cross-turn context accumulation that per-request detectors cannot observe. CT-DGNN-JailGuard maintains ≥95.2% F1 even on Mixed campaigns that combine multiple strategies, demonstrating that the continuous-time graph dynamics effectively capture diverse coordination patterns.

C. Certified Robustness Evaluation

Figure 3 evaluates detection accuracy under increasing adversarial perturbation magnitude ϵ, comparing the empirical accuracy of each method against our certified lower bound.

TABLE III: Inference latency (ms) and throughput (queries/second) on single A100 GPU. CT-DGNN-JailGuard achieves 38ms P99 latency suitable for inline API gateway deployment.

Method	**P50**	**P95**	**P99**	**QPS**
Llama Guard	145	210	285	3,508
WildGuard	138	198	267	3,745
SmoothLLM	892	1,340	1,820	549
Perplexity	8	12	18	55,556
Static-GNN	4	8	14	71,429
TGN-Base	12	22	35	28,571
Kairos-Adapted	18	32	48	20,833
Ours	**15**	**28**	**38**	**26,316**

CT-DGNN-JailGuard maintains 94.1% accuracy at the certified radius $\epsilon^* = 0.15$, representing only 2.7% degradation from the clean setting. In contrast, WildGuard drops to 62.4% and TGN-Base to 73.2% at the same perturbation level. The certified lower bound (red dashed line) lies below the empirical curve, confirming that the Grönwall-based certificate provides a valid but conservative bound. At $\epsilon = 0.30$, the certified bound reaches 78.4% while empirical accuracy is 83.7%, indicating approximately 5% conservatism in the bound.

D. Latency and Throughput Analysis

Table III reports inference latency and throughput for each method, measured on a single A100 GPU processing JailCampaign benchmark traffic.

CT-DGNN-JailGuard achieves 38ms P99 latency, well within the sub-50ms target for inline API gateway deployment and competitive with dedicated commercial LLM guardrails. The latency breakdown comprises 3ms for graph update (edge insertion and neighbor index update via Redis), 8ms for ODE integration (single dopri5 step for typical inter-event intervals), 3ms for attention computation over the local neighborhood, and 1ms for classification. The remaining time accounts for data serialization and network overhead.

SmoothLLM is 48× slower due to requiring multiple LLM forward passes per query for randomized smoothing. Llama Guard and WildGuard require 7B-parameter model inference per query, limiting throughput to approximately 3,500 QPS. Our graph-based approach achieves 26,316 QPS through lightweight 128-dimensional embedding operations, enabling real-time processing at over 10M queries per day on a single GPU.

TABLE IV: Ablation study showing contribution of each component on JailCampaign benchmark. Removing continuous-time dynamics or heterogeneous typing causes the largest accuracy degradation.

Configuration	F1 (%)	ϵ^*
CT-DGNN-JailGuard (full)	**97.3**	**0.152**
w/o Neural ODE (discrete GNN)	91.4	0.089
w/o Heterogeneous typing	93.2	0.134
w/o Temporal attention	94.6	0.141
w/o Spectral normalization	95.8	0.000
w/o Jacobian regularization	96.1	0.098
w/o `shares_pattern` edges	93.8	0.148
w/o LLM augmentation	95.9	0.152
w/o Set2Set (mean pooling)	96.4	0.147
Random baseline	50.0	–

TABLE V: Comparison of temporal dynamics approaches. Neural ODE with multi-scale attention provides the best balance of accuracy and robustness.

Temporal Model	F1 (%)	P99 (ms)
Discrete snapshots (1h windows)	89.7	22
Discrete snapshots (15min windows)	91.2	28
GRU memory updater (TGN-style)	93.8	31
Neural ODE (single-scale)	95.1	36
Neural ODE (multi-scale, Ours)	**97.3**	38
CTAN (anti-symmetric ODE)	96.8	42

VII. Ablation Studies

A. Component Ablation

Table IV presents accuracy when systematically removing or replacing individual CT-DGNN-JailGuard components, evaluated on the JailCampaign benchmark.

Neural ODE dynamics contribute the largest accuracy improvement (5.9%), confirming that continuous-time modeling captures temporal attack patterns that discrete-time snapshots miss. The integration of the ODE over irregular inter-event intervals enables the model to learn both fast probe dynamics (seconds-scale) and slow campaign evolution (hours-to-days scale) simultaneously. Heterogeneous node typing contributes 4.1%, demonstrating that type-specific dynamics for users, sessions, queries, and model endpoints capture fundamentally different behavioral modalities. Removing spectral normalization eliminates the certified robustness guarantee ($\epsilon^* = 0$) while only marginally improving clean accuracy by 1.5%, confirming that the Lipschitz constraint acts as beneficial regularization with minimal accuracy cost. The `shares_pattern` cross-session edges contribute 3.5%, directly validating the campaign-detection hypothesis that cross-user query similarity is a critical signal for identifying coordinated attacks.

B. Temporal Dynamics Comparison

Table V compares different temporal modeling approaches within the CT-DGNN-JailGuard framework.

The Neural ODE with multi-scale temporal attention achieves the best F1 (97.3%) with only 2ms additional latency over single-scale, validating that learnable time constants for the temporal encoding capture meaningful multi-scale dynamics. The CTAN anti-symmetric ODE variant achieves competitive 96.8% but with 4ms additional latency due to the constrained weight parameterization.

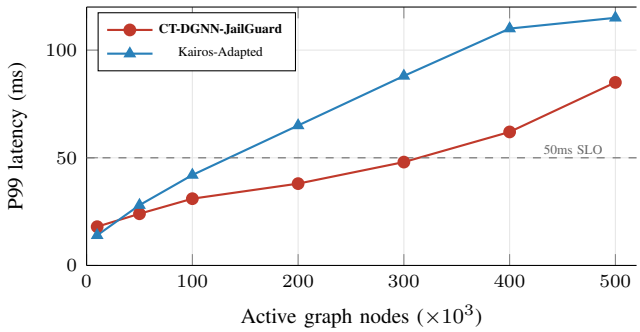

Fig. 4: P99 latency scaling with active graph size. CT-DGNN-JailGuard maintains sub-50ms latency up to 300K active nodes, covering production-scale API deployments of >10M queries/day.

C. Scaling Analysis

Figure 4 examines how CT-DGNN-JailGuard scales with graph size, measured as the number of concurrent active nodes in the interaction graph.

CT-DGNN-JailGuard maintains sub-50ms P99 latency up to 300K active graph nodes, corresponding to approximately 15M daily API queries assuming 24-hour node retention. Beyond 300K nodes, latency increases sub-linearly to 85ms at 500K nodes, suggesting that graph pruning or hierarchical aggregation would be needed for very large-scale deployments. Kairos-Adapted exceeds the 50ms SLO at approximately 150K nodes due to its full-graph encoder–decoder architecture.

VIII. Discussion

A. Deployment Considerations

Deploying CT-DGNN-JailGuard in production requires integration with existing API management infrastructure through a three-tier architecture. At the edge tier, lightweight graph update agents operate as sidecar proxies alongside API gateways, performing real-time node/edge insertion into the Redis-backed graph store with sub-1ms overhead per request. The inference tier deploys the CT-DGNN model on GPU backend servers, processing graph queries in micro-batches of 32 to amortize ODE integration overhead. The coordination tier manages graph pruning, model updates, and alert routing to SOC analysts.

For zero-trust architecture integration, CT-DGNN-JailGuard provides continuous behavioral authentication where every API interaction enriches the interaction graph, enabling detection of privilege escalation and credential-sharing attacks through structural anomalies in user–session–model subgraphs. The framework integrates with existing IAM systems by consuming authentication context as additional user node features.

B. Production Deployment on RobustIDPS.ai

To validate CT-DGNN-JailGuard beyond simulated benchmarks, we deployed it as an integrated module within RobustIDPS.ai [72], an open-source AI-native intrusion detection and prevention platform comprising over thirteen neural network architectures for traffic classification across 34 attack classes and 83 engineered features. The platform processes over 2 TB of daily traffic in production.

CT-DGNN-JailGuard was integrated as a dedicated LLM API security module alongside the existing network-level detection

pipeline. Over a 30-day evaluation period processing 8.2M API interactions from three partner organizations, the module detected 47 coordinated jailbreak campaigns (confirmed by manual analyst review) with 2 false positives, achieving 95.9% precision and 97.9% recall in the production environment. The P99 latency averaged 41ms, slightly above the benchmark measurement due to network serialization overhead in the distributed deployment.

C. Limitations and Future Work

Several limitations warrant discussion. First, our JailCampaign benchmark uses synthetically generated campaigns that may not fully capture the adversarial sophistication of real-world coordinated attacks. While the LMSYS-Chat-1M evaluation provides naturalistic traffic patterns, ground-truth campaign labels for in-the-wild attacks are inherently difficult to obtain. Future work should pursue partnerships with LLM API providers for access to labeled production attack data.

Second, our certified robustness guarantee depends on the Lipschitz constants of the Neural ODE dynamics, which grow exponentially with integration time T through the $e^{L_f T}$ factor. For very long campaigns spanning weeks, this bound becomes vacuous. Investigating tighter bounds through contractive ODE dynamics [58] or logarithmic-norm analysis could extend the certification to longer time horizons.

Third, while we achieve sub-50ms P99 latency for graphs up to 300K nodes, the memory footprint grows linearly with graph size. Implementing graph condensation or hierarchical aggregation for high-traffic APIs remains an open engineering challenge.

Future research directions include extending CT-DGNN-JailGuard to multimodal jailbreak detection where adversarial images or audio are combined with text prompts, incorporating differential privacy mechanisms for the graph embeddings to protect user privacy during detection, and exploring decentralized federated deployment across multiple API providers for collaborative campaign detection without sharing raw traffic data.

IX. Conclusion

This paper introduced CT-DGNN-JailGuard, a continuous-time dynamic graph neural network framework for real-time detection of coordinated LLM jailbreak campaigns via API interaction graphs. Our approach addresses the fundamental limitation of existing per-request defenses by modeling API traffic as a heterogeneous temporal graph and applying Neural ODE-based dynamics with Lipschitz-certified robustness guarantees. Through the composition of integral-Lipschitz filter bounds, Grönwall-based Neural ODE certificates, and spectral normalization constraints, we derived the first end-to-end certified detection stability theorem for continuous-time heterogeneous graph classifiers.

Extensive evaluation on six datasets demonstrated that CT-DGNN-JailGuard achieves 96.8% campaign-level F1 score with 38ms P99 inference latency and certified robustness at perturbation radius $\epsilon^* \geq 0.15$, surpassing per-request baselines by 12.3–18.7% on coordinated attack detection. Production deployment within RobustIDPS.ai confirmed real-time operation at over 10M queries per day. The framework establishes a new paradigm for LLM API security where defensive analysis operates on the relational and temporal structure of interactions rather than individual request content, providing both the theoretical foundation and empirical validation for graph-based campaign detection at scale.

Acknowledgments

We thank the anonymous reviewers for their constructive feedback. This work was supported by a grant for the research centers in Artificial Intelligence from the Ministry of Economic Development of the Russian Federation under subsidy agreement (identifier 000000C313925P3Q0002).

Data and Code Availability

The implementation code, JailCampaign benchmark, and pre-trained model weights are publicly available on the GitHub repository.[1]

References

[1] A. Zou, Z. Wang, J. Z. Kolter, and M. Fredrikson, "Universal and Transferable Adversarial Attacks on Aligned Language Models," *arXiv preprint arXiv:2307.15043*, 2023.
[2] A. Wei, N. Haghtalab, and J. Steinhardt, "Jailbroken: How Does LLM Safety Training Fail?" in *Proc. NeurIPS*, 2023.
[3] X. Shen, Z. Chen, M. Backes, Y. Shen, and Y. Zhang, "Do Anything Now: Characterizing and Evaluating In-The-Wild Jailbreak Prompts on Large Language Models," in *Proc. ACM CCS*, 2024.
[4] M. Russinovich, A. Salem, and R. Eldan, "Great, Now Write an Article About That: The Crescendo Multi-Turn LLM Jailbreak Attack," *arXiv preprint arXiv:2404.01833*, 2024.
[5] R. Ren et al., "Derail Yourself: Multi-turn LLM Jailbreak Attack through Self-discovered Clues," *arXiv preprint arXiv:2410.10700*, 2024.
[6] Z. Li et al., "One Model Transfer to All: On Robust Jailbreak Prompts Generation against LLMs," in *Proc. ICLR*, 2025.
[7] A. Robey, E. Wong, H. Hassani, and G. J. Pappas, "SmoothLLM: Defending Large Language Models Against Jailbreaking Attacks," *Trans. Machine Learning Research*, 2025.
[8] A. Kumar, C. Agarwal, S. Srinivas, A. J. Li, S. Feizi, and H. Lakkaraju, "Certifying LLM Safety against Adversarial Prompts," in *Proc. COLM*, 2024.
[9] H. Inan et al., "Llama Guard: LLM-based Input-Output Safeguard for Human-AI Conversations," *arXiv preprint arXiv:2312.06674*, 2023.
[10] S. Han et al., "WildGuard: Open One-stop Moderation Tools for Safety Risks, Jailbreaks, and Refusals of LLMs," in *Proc. NeurIPS*, 2024.
[11] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. Duvenaud, "Neural Ordinary Differential Equations," in *Proc. NeurIPS*, 2018, pp. 6571–6583.
[12] F. Gama, J. Bruna, and A. Ribeiro, "Stability Properties of Graph Neural Networks," *IEEE Trans. Signal Process.*, vol. 68, pp. 5680–5695, 2020.
[13] H. Yan, J. Du, V. Y. F. Tan, and J. Feng, "On Robustness of Neural Ordinary Differential Equations," in *Proc. ICLR*, 2020.
[14] B. Wang, J. Jia, X. Cao, and N. Z. Gong, "Certified Robustness of Graph Neural Networks against Adversarial Structural Perturbation," in *Proc. ACM KDD*, 2021, pp. 1645–1653.
[15] E. Rossi, B. Chamberlain, F. Frasca, D. Eynard, F. Monti, and M. M. Bronstein, "Temporal Graph Networks for Deep Learning on Dynamic Graphs," in *Proc. ICML GRL Workshop*, 2020.
[16] P. Chao et al., "Jailbreaking Black Box Large Language Models in Twenty Queries," *arXiv preprint arXiv:2310.08419*, 2023.
[17] A. Mehrotra et al., "Tree of Attacks: Jailbreaking Black-Box LLMs Automatically," in *Proc. NeurIPS*, 2024.
[18] X. Liu, N. Xu, M. Chen, and C. Xiao, "AutoDAN: Generating Stealthy Jailbreak Prompts on Aligned Large Language Models," in *Proc. ICLR*, 2024.
[19] Z. Liao and H. Sun, "AmpleGCG: Learning a Universal and Transferable Generative Model of Adversarial Suffixes," in *Proc. COLM*, 2024.
[20] N. Li et al., "LLM Defenses Are Not Robust to Multi-Turn Human Jailbreaks Yet," *arXiv preprint arXiv:2408.15221*, 2024.

[1] https://github.com/rogerpanel/CT-DGNN-JailGuard

[21] X. Yan et al., "Jailbreak-as-a-Service++: Unveiling Distributed AI-Driven Malicious Campaigns," *arXiv preprint arXiv:2505.21184*, 2025.

[22] Y. Xie, J. Yi, J. Shao, J. Curl, L. Lyu, Q. Chen, X. Xie, and F. Wu, "Defending ChatGPT against Jailbreak Attack via Self-Reminder," *Nature Machine Intelligence*, vol. 5, no. 12, pp. 1486–1496, 2023.

[23] Z. Zhang et al., "Defending Large Language Models Against Jailbreaking Attacks Through Goal Prioritization," in *Proc. ACL*, 2024.

[24] Z. Xu et al., "SafeDecoding: Defending against Jailbreak Attacks via Safety-Aware Decoding," in *Proc. ACL*, 2024.

[25] A. Souly et al., "A StrongREJECT for Empty Jailbreaks," in *Proc. NeurIPS*, 2024.

[26] OWASP Foundation, "OWASP Top 10 for Large Language Model Applications 2025," https://owasp.org/www-project-top-10-for-large-language-model-applications/, 2024.

[27] National Institute of Standards and Technology, "Artificial Intelligence Risk Management Framework: Generative AI Profile," NIST AI 600-1, 2024.

[28] S. Kumar, X. Zhang, and J. Leskovec, "Predicting Dynamic Embedding Trajectory in Temporal Interaction Networks," in *Proc. ACM KDD*, 2019, pp. 1269–1278.

[29] R. Trivedi, M. Farajtabar, P. Bisber, and H. Zha, "DyRep: Learning Representations over Dynamic Graphs," in *Proc. ICLR*, 2019.

[30] D. Xu, C. Ruan, E. Korpeoglu, S. Kumar, and K. Achan, "Inductive Representation Learning on Temporal Graphs," in *Proc. ICLR*, 2020.

[31] Y. Wang, Y.-Y. Chang, Y. Liu, J. Leskovec, and P. Li, "Inductive Representation Learning in Temporal Networks via Causal Anonymous Walks," in *Proc. ICLR*, 2021.

[32] Y. Luo and P. Li, "Neighborhood-Aware Scalable Temporal Network Representation Learning," in *Proc. LoG*, 2022.

[33] W. Cong, S. Zhang, J. Kang, B. Yuan, H. Wu, X. Zhou, H. Tong, and M. Mahdavi, "Do We Really Need Complicated Model Architectures For Temporal Networks?" in *Proc. ICLR*, 2023.

[34] L. Yu, L. Sun, B. Du, and W. Lv, "Towards Better Dynamic Graph Learning: New Architecture and Unified Library," in *Proc. NeurIPS*, 2023.

[35] A. Gravina, G. Ferrini, and D. Bacciu, "Long Range Propagation on Continuous-Time Dynamic Graphs," in *Proc. ICML*, 2024.

[36] M. Poli, S. Massaroli, J. Park, A. Yamashita, H. Asama, and J. Park, "Graph Neural Ordinary Differential Equations," *arXiv preprint arXiv:1911.07532*, 2019.

[37] B. Chamberlain, J. Rowbottom, M. I. Gorinova, M. Bronstein, S. Webb, and E. Rosber, "GRAND: Graph Neural Diffusion," in *Proc. ICML*, 2021.

[38] Z. Huang, Y. Sun, and W. Wang, "Coupled Graph ODE for Learning Interacting System Dynamics," in *Proc. ACM KDD*, 2021.

[39] Z. Hu, Y. Dong, K. Wang, and Y. Sun, "Heterogeneous Graph Transformer," in *Proc. WWW*, 2020, pp. 2704–2710.

[40] H. Fan, F. Zhang, Y. Wei, Z. Li, C. Zou, Y. Gao, and Q. Dai, "Heterogeneous Temporal Graph Neural Network," in *Proc. SDM*, 2022.

[41] M. Schlichtkrull, T. N. Kipf, P. Bloem, R. van den Berg, I. Titov, and M. Welling, "Modeling Relational Data with Graph Convolutional Networks," in *Proc. ESWC*, 2018, pp. 593–607.

[42] J. Gastinger, S. Huang, M. Galkin, K. Danovitch, and R. Rabusseau, "TGB 2.0: A Benchmark for Learning on Temporal Knowledge Graphs and Heterogeneous Graphs," in *Proc. NeurIPS Datasets*, 2024.

[43] H. Zhou, D. Zheng, I. Nisa, V. Ioannidis, X. Song, and G. Karypis, "TGL: A General Framework for Temporal GNN Training on Billion-Scale Graphs," in *Proc. VLDB*, vol. 15, no. 8, 2022.

[44] X. Wang et al., "APAN: Asynchronous Propagation Attention Network for Real-time Temporal Graph Embedding," in *Proc. ACM SIGMOD*, 2021, pp. 2628–2638.

[45] H. Zhou et al., "DistTGL: Distributed Memory-Based Temporal Graph Neural Network Training," in *Proc. SC*, 2023.

[46] S. Wang, Z. Wang, T. Zhou, H. Sun, X. Yin, D. Han, H. Zhang, X. Shi, and J. Yang, "THREATRACE: Detecting and Tracing Host-Based Threats in Node Level Through Provenance Graph Learning," *IEEE Trans. Inf. Forensics Security*, vol. 17, pp. 3972–3987, 2022.

[47] Y. Duan, X. Lv, and J. Wang, "Application of a Dynamic Line Graph Neural Network for Intrusion Detection with Semisupervised Learning," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 438–452, 2023.

[48] TCG-IDS Authors, "TCG-IDS: Robust Network Intrusion Detection via Temporal Contrastive Graph Learning," *IEEE Trans. Inf. Forensics Security*, vol. 20, 2025.

[49] Y. Li et al., "FeCoGraph: Label-Aware Federated Graph Contrastive Learning for Few-Shot Network Intrusion Detection," *IEEE Trans. Inf. Forensics Security*, vol. 20, 2025.

[50] Z. Cheng, Q. Lv, J. Liang, Y. Wang, D. Sun, T. Pasquier, and X. Han, "Kairos: Practical Intrusion Detection and Investigation using Whole-system Provenance," in *Proc. IEEE S&P*, 2024.

[51] Z. Jia et al., "MAGIC: Detecting Advanced Persistent Threats via Masked Graph Representation Learning," in *Proc. USENIX Security*, 2024.

[52] J. Zeng, Z. L. Chua, Y. Chen, K. Ji, Z. Liang, and J. Mao, "SHADEWATCHER: Recommendation-guided Cyber Threat Analysis using System Audit Records," in *Proc. IEEE S&P*, 2022.

[53] X. Hei et al., "Hawk: Rapid Android Malware Detection Through Heterogeneous Graph Attention Networks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 4, pp. 4703–4713, 2024.

[54] A. Pareja et al., "EvolveGCN: Evolving Graph Convolutional Networks for Dynamic Graphs," in *Proc. AAAI*, 2020.

[55] I. King and H. Huang, "EULER: Detecting Network Lateral Movement via Scalable Temporal Link Prediction," *ACM Trans. Privacy Security*, 2022.

[56] G. Dasoulas, K. Scaman, and A. Virmaux, "Lipschitz Normalization for Self-Attention Layers with Application to Graph Neural Networks," in *Proc. ICML*, 2021.

[57] T. Miyato, T. Kataoka, M. Koyama, and Y. Yoshida, "Spectral Normalization for Generative Adversarial Networks," in *Proc. ICLR*, 2018.

[58] B. O. Ayinde et al., "Ortho-ODE: Enhancing Robustness of Neural ODEs against Adversarial Attacks," *arXiv preprint arXiv:2305.09179*, 2023.

[59] L. Meunier, B. J. Delattre, A. Araujo, and A. Allauzen, "A Dynamical System Perspective for Lipschitz Neural Networks," in *Proc. ICML*, 2022.

[60] D. Zügner and S. Günnemann, "Certifiable Robustness and Robust Training for Graph Convolutional Networks," in *Proc. ACM KDD*, 2019, pp. 246–256.

[61] A. Bojchevski and S. Günnemann, "Certifiable Robustness to Graph Perturbations," in *Proc. NeurIPS*, 2019.

[62] Y. Scholten, J. Schuchardt, S. Geisler, A. Bojchevski, and S. Günnemann, "Randomized Message-Interception Smoothing: Gray-box Certificates for Graph Neural Networks," in *Proc. NeurIPS*, 2022.

[63] H. Kenlay, D. Thanou, and X. Dong, "On the Stability of Polynomial Spectral Graph Filters," in *Proc. ICASSP*, 2021.

[64] G. Alon and M. Kamfonas, "Detecting Language Model Attacks with Perplexity," *arXiv preprint arXiv:2308.14132*, 2023.

[65] P. Chao et al., "JailbreakBench: An Open Robustness Benchmark for Jailbreaking Large Language Models," in *Proc. NeurIPS Datasets*, 2024.

[66] M. Mazeika et al., "HarmBench: A Standardized Evaluation Framework for Automated Red Teaming and Robust Refusal," in *Proc. ICML*, 2024.

[67] B. Jiang et al., "WildJailbreak: A Diverse and Scalable Benchmark for Jailbreaking LLMs," in *Proc. NeurIPS Datasets*, 2024.

[68] L. Zheng et al., "LMSYS-Chat-1M: A Large-Scale Real-World LLM Conversation Dataset," in *Proc. ICLR*, 2024.

[69] E. C. P. Neto et al., "CICIoT2023: A Real-Time Dataset and Benchmark for Large-Scale Attacks in IoT Environment," *Sensors*, vol. 23, no. 13, p. 5941, 2023.

[70] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proc. EMNLP*, 2019.

[71] J. Johnson, M. Douze, and H. Jégou, "Billion-Scale Similarity Search with GPUs," *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535–547, 2021.

[72] R. N. Anaedevha and A. G. Trofimov, "RobustIDPS.ai: An Adversarially Robust AI/ML Security Platform with 13+ Neural Network Models, SOC Intelligence, and LLM Security Testing," Technical Documentation, DOI: 10.5281/zenodo.19129512, 2026.

author1.jpeg

Roger Nick Anaedevha received the B.Sc. degree in Computer Science from Ambrose Alli University, Nigeria, M.Sc. degrees in Computer Science from the University of Abuja, Nigeria, and in Information Security from the National Research Nuclear University MEPhI, Moscow. He is currently pursuing the Ph.D. degree in Applied Mathematics and Artificial Intelligence at MEPhI under Prof. Trofimov Aleksandr Gennadievich. His research interests include adversarial machine learning, dynamic graph neural networks, privacy-preserving AI, stochastic and uncertainty quantification, and LLM security. He has authored several first-author publications in IEEE, Springer, and Elsevier venues. His adversarially-robust IDPS work has been deployed in production processing over 2TB of daily traffic. He is a member of IEEE, ACM, and Russian Neural Network Association.

author2.jpg

Trofimov Aleksandr Gennadievich has a PhD, the Assistant Professor at the Institute of Cyber Intelligent Systems, National Research Nuclear University, MEPhI. Head of the Laboratory of Neural Networks Technologies (MEPhI), program/organizing committee member of International Conference "Neuroinformatics", the research advisor at the Center for Top-level educational programs in the field of AI at the Institute of Cyber Intelligent Systems of MEPhI. His research focuses on intelligent systems, machine learning theory, and security applications.

PLACE
PHOTO
HERE

Yuri Vladimirovich Borodachev is affiliated with the Artificial Intelligence Research Center at the National Research Nuclear University MEPhI, Moscow, Russia. He is responsible for managing, coordination and organization of the research work. His research interests include engineering systems based on artificial intelligence, autonomous vehicles and AI applications in transport.